

# User Manual: A.R.T.E. System & Modules

**Lead Architect: Emanuel Schaaf | Open Origin Architecture**

This manual describes the correct startup and initialization of the A.R.T.E. System, the loading of its associated modules, and the fundamental guidelines for new developments and utilizing the App Store.

---

## 1. System Startup (Strict Sequence)

The A.R.T.E. System requires a strict boot sequence. The core system must always be loaded first.

1. Navigate to the `./System` folder on your computer.
  2. Open the file `ARTE_System_Core.html` in a modern web browser (e.g., by double-clicking the file).
  3. Wait until the "Open Origin - OS Control Room" dashboard is fully loaded and the status displays "Ready".
- 

## 2. Loading Modules

Once the system core is running, the desired modules must be injected into the system from the `/Module` folder. There are two ways to do this:

### **Method A: Drag & Drop (Recommended)**

1. Open the file explorer and navigate to the `/Module` folder.
2. Select the desired HTML module files.
3. Drag the files with your mouse and drop them directly into the dashed "Drop-Zone" in the ARTE Core Dashboard (marked "Drag modules here").

### **Method B: Manual Upload**

1. Click on the "Drop-Zone" in the ARTE Core Dashboard.
2. A file selection dialog will open.
3. Navigate to the `/Module` folder, select the desired HTML files, and confirm the upload.

 **Note:** The system will now read the modules and make them available as new, interactive tabs in the "Workspace Layer" (bottom tab bar).

---

### 3. The A.R.T.E.App-Panel.html (Simple App Store)

The `A.R.T.E.App-Panel.html` module is a revolutionary sandbox environment within the system. It acts as a local app store that allows you to load, manage, and program micro-apps in isolation.

#### Operation & App Management

- **Loading External Apps:** Drag and drop HTML app files into the dedicated store zone. They are immediately saved in a local database ( `IndexedDB` ) and installed as dynamic tabs in the top bar.
- **Tabs & Navigation:** Apps line up seamlessly as tabs (e.g., App 1, App 2). When an app is closed via the red **X** , the remaining apps automatically adjust their numbering.
- **Auto-Heal I-Frame:** To prevent "white-screen" rendering errors (display-none death), the system destroys the old I-Frame upon tab switching and generates a fresh one.
- **Export:** Any running app can be saved as a complete, standalone `.html` file using the "Export" button.

#### Terminal Node (Write your own apps)

Click the "Terminal Node" tab to open the integrated code editor. Here, custom HTML-based apps can be written and compiled directly into the store.

#### Important Guidelines for Writing Custom Apps:

1. **HTML Focus:** Only HTML-based code in raw format is accepted.
  2. **Dimensions:** The app should adapt fluidly, as the system attempts to automatically fit it into the right workspace area (I-Frame).
  3. **Auto-i18n Injection:** If your code is missing the connection to the system language bus ( `window.osBus.on('lang-changed')` ), the Terminal automatically injects this translation logic into the background of your code upon saving. The app is immediately system-compatible afterwards.
  4. **Save & Confirm:** After writing, the code must be confirmed by clicking " Compile & Save App". The app will then immediately appear in the tab menu. When exporting later, the injected system translation logic will be permanently anchored in the code.
-

## 4. Overview of Available Modules

The following modules can be loaded and used after system startup:

-  **A.R.T.E.App-Panel.html**: The local Simple App Store featuring dynamic tabs, I-Frame isolation, and an integrated Terminal Node for app development.
  -  **ARTE\_Currency\_Core.html**: A self-sufficient currency conversion matrix with a live satellite uplink for current rates and an integrated offline fallback database for maximum reliability.
  -  **ARTE\_Sci\_Calc.html**: A scientific calculator featuring Kinetic UI and safe eval routing. Computes complex mathematical formulas (sin, cos, log, etc.) inside a strictly isolated sandbox.
  -  **A.R.T.E. Quantum Canvas**: A hybrid raster/vector drawing environment featuring dynamic workspace expansion, a symmetry matrix, rotation tools, and a synchronous render engine for flawless exports.
  -  **ARTE\_Chronos\_Matrix.html**: A spherical time engine module with Kinetic UI, hardware canvas rendering, and global timezone synchronization, including persistent local storage.
  -  **Audio\_Analyser-Pipeline.html (Acoustic Resonance Engine)**: Visualizes audio files in real-time with reactive effects (e.g., Cyberpunk Matrix, Nebula).
  -  **Image\_Resizer\_Matrix (High Fidelity Scaler)**: Enables precise image scaling while maintaining proportions, including lossless export (PNG, JPG, WEBP).
  -  **Kinetic Telemetry&\_Canvas\_Engine.html**: A GPU-accelerated oscilloscope for real-time data visualization on an HTML5 canvas.
  -  **Optical Contour&\_Blueprint\_Synthesizer.html**: Converts images locally via edge detection into detailed blueprints.
  -  **Kinetic Capture & Imaging Engine**: Connects the system to video feeds for snapshots and direct image editing in the focus room.
  -  **Advanced Document Synthesis Matrix**: Local WYSIWYG text editor with A4 paper simulation and PDF/HTML/MD/TXT export.
  -  **Universal Media Converter**: Transcodes media offline in the browser (MP3, MP4, WebM, M4A).
  -  **Kinetic Video Engine**: Allows loading videos for real-time playback, cropping, and codec export.
  -  **Global Weather Radar / 48H Forecast**: Interactive weather map with radar animations.
  -  **Live Web Synthesis Matrix**: Browser IDE for HTML/CSS/JS with live preview.
-

## 5. Development Guidelines for New Modules

When developing new A.R.T.E. modules, the following architecture and content standards must be strictly adhered to in order to ensure system stability:

- **Shadow DOM Isolation:** For security reasons, every module is loaded from the core system into its own isolated Shadow DOM. Global queries like `document.getElementById()` will fail. A dynamic root reference must be formed (e.g., `const root = typeof shadowRoot !== 'undefined' ? shadowRoot : document;`).
- **Local i18n & Event Bus (osBus):** Modules must operate fully bilingual (German/English). The module must listen for language changes via the global event bus ( `window.osBus.on('lang-changed', ...)` ) to update the UI dynamically without reloading.
- **Scientific Precision & Functionality:** All calculations and processes must be mathematically precise, traceable, and physically and chemically consistent. Unsubstantiated assumptions are to be avoided.
- **Universal Utility & Strict Neutrality:** The code and user interface must offer the potential for broad, general utility. The inclusion of any political statements, judgments, or biases is strictly prohibited.